ARCHITECTURE BRIEF

The SKYCOT Protocol

A reusable protocol for coordinating a swarm of autonomous AI coding agents on a single git repository — using one shared file, no servers, and the plain rules of `git push`.

22	~21	~60 min	100%
BRIEF SLICES	PRS MERGED	WALL-CLOCK	TESTS GREEN

Bootstrapped on the SKYCOT v1 build of `argo` · 28 crates touched · every file under 500 LOC.

Mark Hallam · Argo / SKYCOT · May 2026

1. The mental model

Three roles, one shared ledger, two output channels. The repository itself becomes the coordination substrate — there is no orchestrator service, no message bus, no database. `git push` is the lock; `BUILD_STATUS.md` is the state.

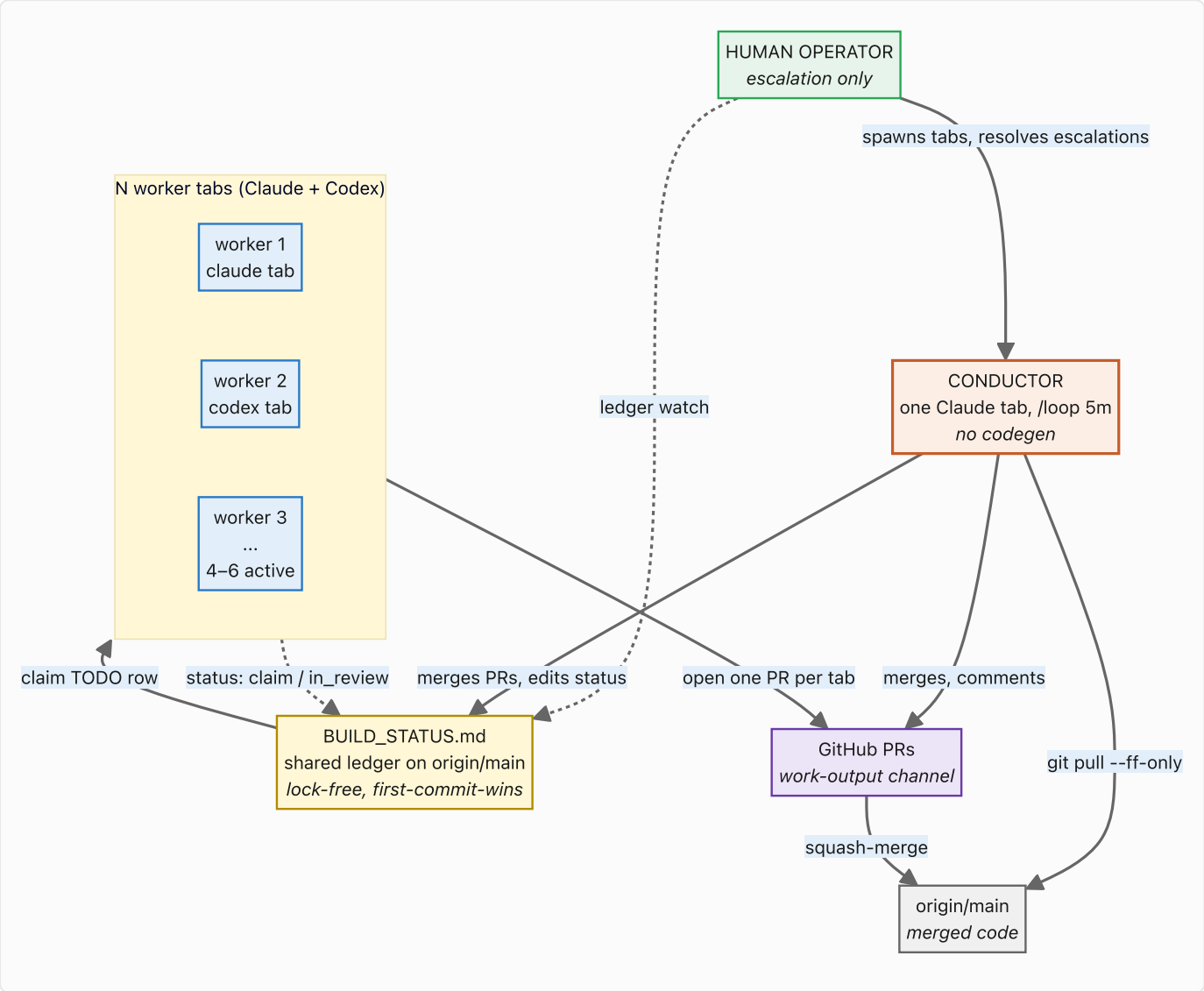


Figure 1. Roles, channels, and read/write arrows. Solid arrows are state transitions; dotted arrows are status pings.

Roles, in detail

Conductor — one Claude tab, /loop 5m

read main · review PRs · merge clean · edit ledger · expand briefs · replenish queue · no feature code · no force push to main

Pulls main, reviews open PRs against each brief's checklist, merges clean ones, comments on issues, unblocks BLOCKED rows, writes fresh briefs when a TODO queue runs shallow (queue replenishment), supervises the swarm. Tick budget is roughly 3 minutes. The only agent allowed to merge.

Worker — N tabs (Claude + Codex)

claim row · implement on worktree · open PR · mark IN_REVIEW · mark BLOCKED · no merging · no scope creep · stop after one PR

One brief per tab, then stop. Branch prefix encodes the agent family (`claude-*`, `codex-*`) so PR provenance is visible at a glance.

Human — escalation target

spawn tabs · resolve mission conflicts · kill runaway tabs

Reviews the rare row the conductor flags. Resolves architectural questions the conductor isn't authorised to make. Watches the ledger periodically.

2. The brief lifecycle state machine

Every row in the ledger lives in exactly one of five states. Transitions are by convention, not code — workers pre-flight on `git pull --ff-only`, the loser pulls and picks a different row. The conductor is the only agent that can merge.

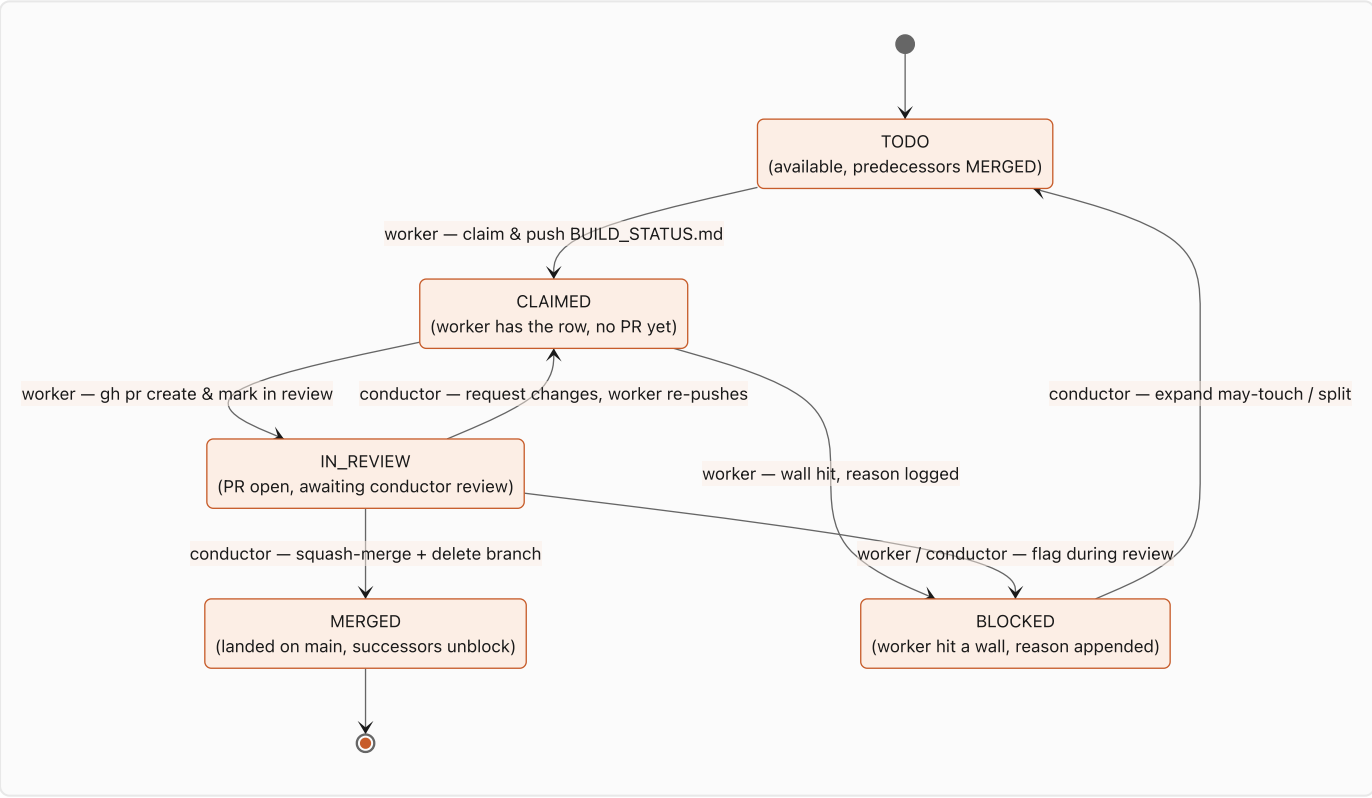


Figure 2. Brief lifecycle. Edge labels name the actor that triggers each transition.

State semantics

State	Meaning	Transition trigger
`TODO`	Available to claim. All *waits-on* predecessors are MERGED.	Worker reads, edits, pushes `status: claim [id]`

State	Meaning	Transition trigger
`CLAIMED`	One worker owns the row. PR not yet open.	Worker pushes `status: [id] in review (#N)`
`IN_REVIEW`	PR open. Conductor will review next tick.	Conductor squash-merges, edits ledger
`MERGED`	Landed on main. Successors may unblock.	Terminal until plan-complete sweep
`BLOCKED`	Worker hit a wall it can't resolve. Reason appended to row.	Conductor expands brief / splits / escalates, then flips back to TODO

3. The conductor tick (5-minute cron)

Nine numbered steps every 5 minutes. About half of ticks are no-ops, about a third merge one PR, the rest merge multiple PRs, unblock a row, or replenish a shallow TODO queue. Roughly 5 percent require human escalation.

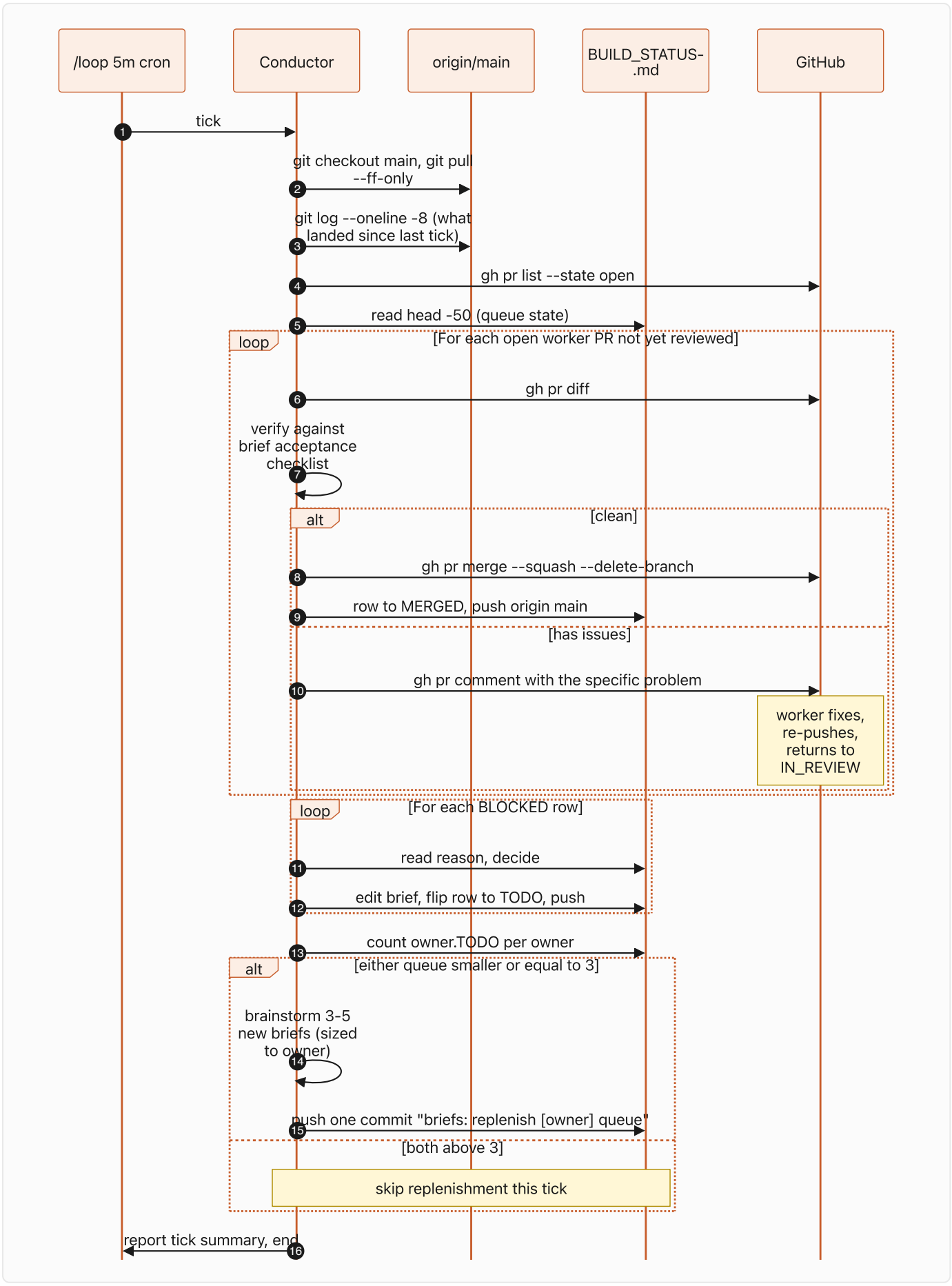


Figure 3. One conductor tick. Notes mark out-of-band feedback to worker tabs (they read on their own next pull). Step 8 (queue replenishment) keeps the swarm fed indefinitely.

Two rules that matter

- **Never write feature code from the conductor tab.** It edits *only* BUILD_STATUS.md and merges PRs (and writes new briefs at step 8 — see section 6). Mixing the conductor with a worker bit the SKYCOT swarm twice — see the webinbox bug in section 7.
- **Always read from a fresh `git pull`**, not a stale local copy. Workers are pushing constantly. The conductor may have a slightly stale local tree; it just must `git pull --ff-only` at the top of every tick before reading or committing anything.

4. The worker tick (one PR, then stop)

The single most important worker rule: **one PR per tab, then stop**. Don't loop. Don't claim a second row. The conductor has visibility on what's mergeable; the worker doesn't.

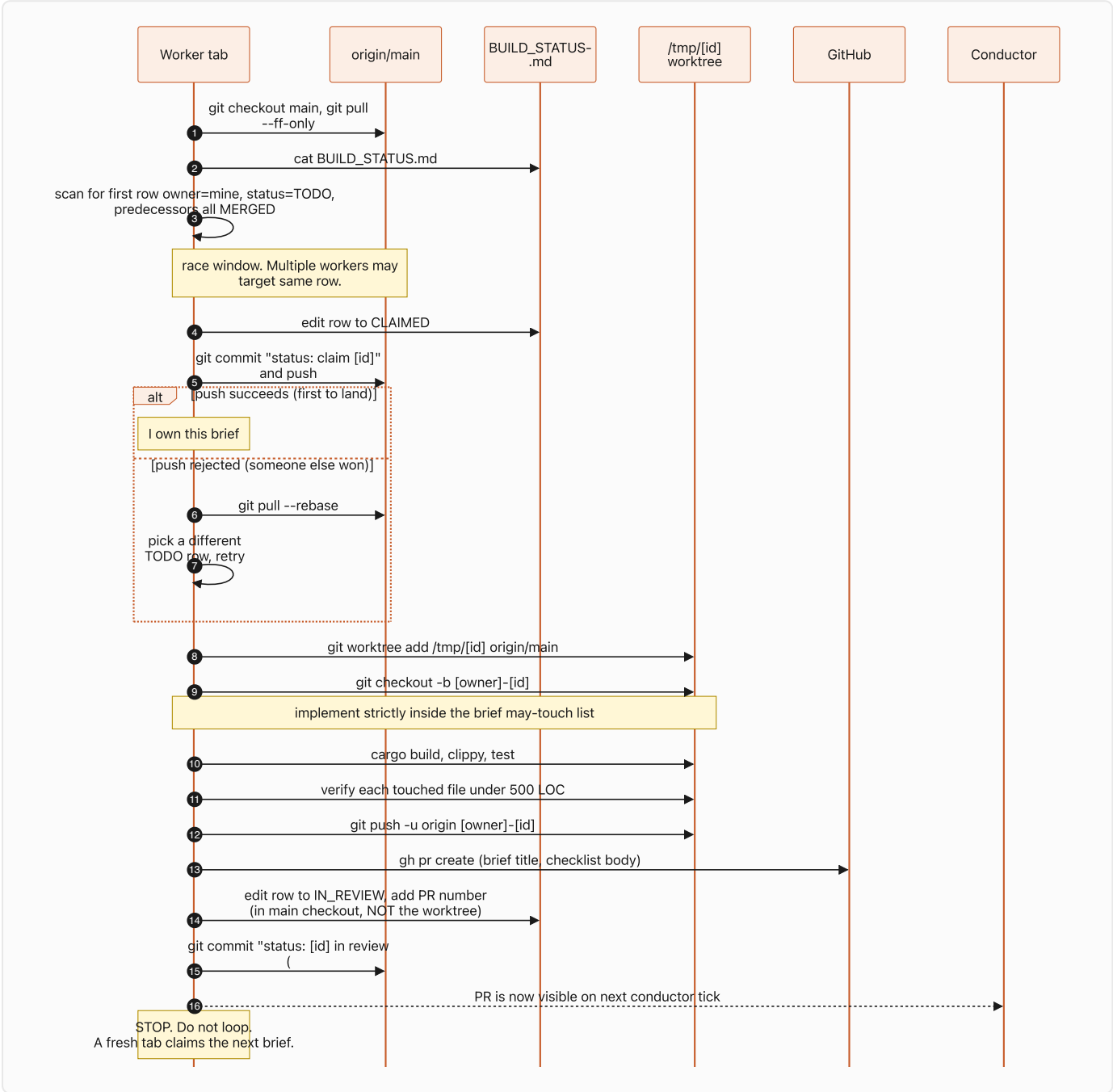


Figure 4. One worker tab, from sync to stop. Race resolution at the CLAIMED step is free: first push wins, loser picks again.

5. The unblock dance

When a worker hits a wall — clippy fails, the may-touch list is too tight, or it discovers a workspace `Cargo.toml` edit the brief forbade — it marks BLOCKED and stops. The conductor reads the reason next tick and decides what to do.

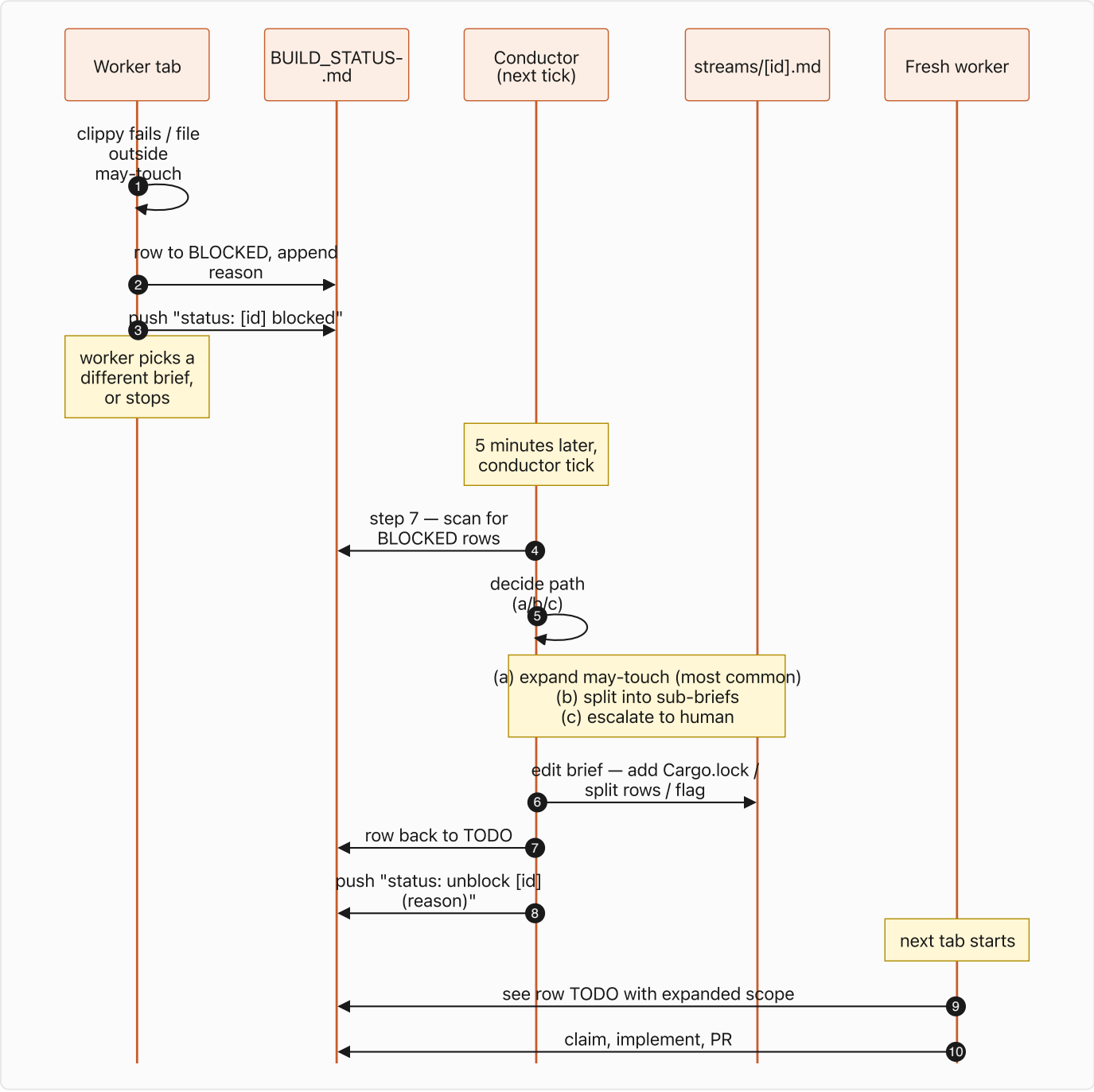


Figure 5. The block-unblock cycle. Typical duration: one or two ticks (5–10 minutes).

Real unblock commits from the SKYCOT run

- `status: unblock C-mcp-wire (Cargo.toml dep add now allowed)`
- `status: unblock C-mcp-wire (Cargo.lock now in may-touch list)`
- `status: unblock all 4 D-* briefs (lib.rs one-line module add allowed)`
- `status: D-push unblocked (use branch codex-D-push-v2)`
- `status: unblock C-gmail (Cargo.lock + defer keyring); apply same Cargo.lock allowance to all 5 C-* connector briefs`

These are the seams the brief author didn't anticipate. Expect one per stream.

6. Queue replenishment — keeping the swarm fed indefinitely

The original SKYCOT plan had a finite queue: 22 briefs across the 14-day v1 build. Once the last brief was MERGED, every worker tab would idle and the swarm would die. **Step 8 of the conductor tick fixes that.** Each cycle the conductor counts TODO rows per owner; if either queue is shallow it brainstorms a fresh batch of briefs and pushes them in one commit. The swarm self-extends.

The trigger fired naturally for the first time when `claude_todo` dropped to 0 — the conductor wrote 4 new claude briefs (`claude-cu-skill-learn-wire` , `claude-tui-h-stack-panels` , `claude-e2e-integration-test` , `claude-genesis-llm-refinement`) and pushed them as a single commit. The queue refilled in one tick.

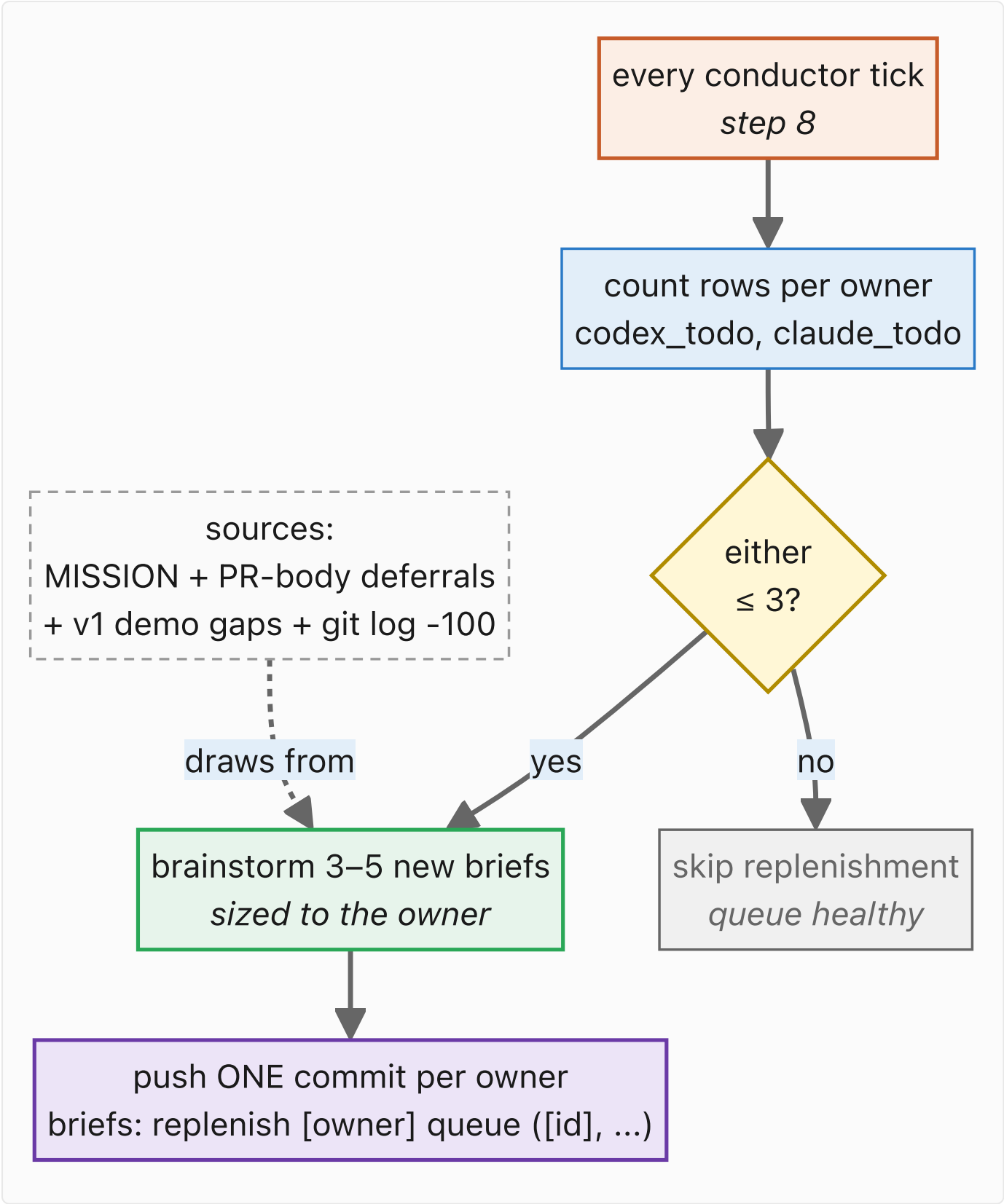


Figure 6. Queue replenishment trigger. Fires every conductor tick; no-op when both queues are healthy.

Idea sources

- `docs/skycot/MISSION.md` — pick a capability that is not yet in main.
- **Recent merged-PR bodies' "deferred to follow-up" sections** — workers routinely flag work they couldn't fit in their brief; the conductor mines those for ready-to-go briefs.
- **The gap between the v1 demo and current main** — what does the demo claim that the code doesn't yet do? Each gap is one brief.

- `git log --oneline origin/main -100` — recent commits often imply natural follow-ups (a new module that needs a test crate, a CLI that needs a JSON flag, a metric that needs a reset).

Sizing — codex vs claude

The two agent families have different strengths. Briefs target the strength of their owner.

Owner	LOC budget	Scope	Acceptance flavor
`codex`	200–1200	Single-crate; narrow may-touch list	Mechanical, well-defined; mirror existing C / D / H briefs
`claude`	300–2000	May span crates; architectural decisions OK	Integration tests welcome; mirror existing claude-* briefs

The push convention

One commit per owner per replenishment tick. Subject line: `briefs: replenish [owner] queue ([id1], [id2], ...)`. The commit edits `BUILD_STATUS.md` (adding new TODO rows) and adds the brief files in `streams/`. Workers see the new rows on their next pull.

Stop condition

> *Replenishment is unbounded by default. The conductor cancels the loop only when **every brief in `BUILD_STATUS.md` is MERGED AND the v1 demo gate is green** — for SKYCOT that means `claude-e2e-integration-test` passing on main and the demo script runnable end-to-end. This represents v1 customer-launch readiness; the conductor knows it's safe to stop because the product can ship.*

7. Race resolutions (lived, not theoretical)

Six anti-patterns hit the SKYCOT run. Each one is a real bug. Recovery cost is in seconds — but only if the protocol anticipates it.

Race 1 — two agents claim the same row

Both pull main, both read TODO, both edit row to CLAIMED, both push. Second push fails fast-forward. Loser pulls, sees row CLAIMED, picks a different row.

Recovery: free, zero human action. Hit ~5 times in 60 minutes; each time worth one extra pull. `status: claim C-mcp-wire` appears 5 times in the log.

Race 2 — duplicate PRs for the same sub-slice

When a brief is sub-sliced (H-1 / H-2 / H-3 / H-4a / H-4b / H-4c), two workers may both think they're claiming H-1. Both PRs land in the queue.

Recovery: conductor's discretion in step 5. Pick the better PR, merge it, close the duplicate with a comment. Lesson: when conductor approves a multi-slice brief, immediately enumerate the slices in the ledger row.

Race 3 — Cargo.toml workspace.members merge conflict

Every new crate edits `members = [...]`. PRs merge in arbitrary order; the second always conflicts on that line.

Recovery: conductor 3-way merges in a worktree. `git fetch && git rebase origin/main && resolve && git push --force-with-lease`. ~30s per conflict, hit five times. Future fix: pre-add empty crate dirs at run start, or lock workspace.members in a single brief.

Race 4 — conductor commit swept up worker WIP (the webinbox bug)

Worker had unstaged webinbox files in the conductor's working tree. Conductor ran `git commit -am "status: …"` with `-a`, accidentally shipped 7 untracked files.

Recovery: row marked MERGED with note; redundant PR closed. Permanent fix: conductor runs from its own worktree on origin/main, AND uses `git add <file> + git commit -m` (never -am). The `PROTOCOL_LOOP_PROMPT.md` explicitly stages only `BUILD_STATUS.md`.

Race 5 — stale branch (codex-D-push)

PR open >2 hours, rebases failing, conflicts piling up. Worker effectively stuck.

Recovery: one-line ledger update authorising a v2 branch. `codex-D-push-v2`; close #785; fresh worker re-implements from main. Lesson: ping any PR open >2h with no progress; close after one more silent tick.

Race 6 — brief shape didn't match reality

Stream A locked the executor schema. C-mcp-wire later needed to add a new `[mcp_servers]` section to argo-vertical, but the brief said "do NOT modify any existing declaration".

Recovery: conductor expanded the brief in two iterations. Lesson: contract briefs (Stream A here) define trait/schema shape; downstream briefs touching that crate should have a "narrow add" clause for new sections baked into the template.

8. The git workflow

Worktree-per-PR is the load-bearing convention. The conductor tab and N worker tabs are checked out on the same repository — separate worktrees mean separate working trees and no `git stash` dance.

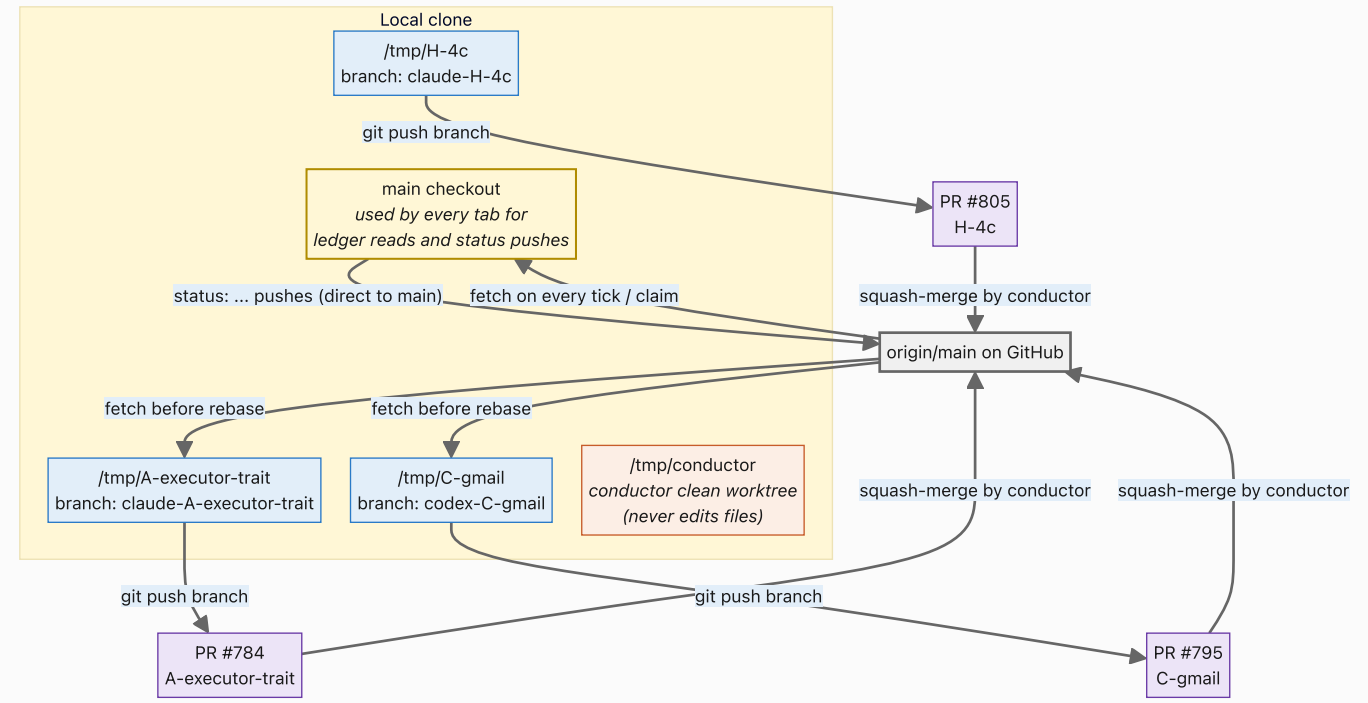


Figure 7. Worktrees on one local clone. Status updates push direct to main; code goes via PR.

The four golden git commands

```
# Worker spin-up — fresh worktree on origin/main (never on stale local)
git worktree add /tmp/[brief-id] origin/main
cd /tmp/[brief-id] && git checkout -b [owner]-[brief-id]
# ... implement, build, test ...
git push -u origin [owner]-[brief-id]

# Worker ledger update — ONLY BUILD_STATUS.md, never -am
git checkout main && git pull --ff-only
# edit BUILD_STATUS.md
git add docs/build-plan/BUILD_STATUS.md
git commit -m "status: [id] in review (#N)"
git push origin main

# Conductor merge — squash + delete branch + preserve brief id in subject
gh pr merge [num] --squash --delete-branch

# Conductor conflict resolution — force-with-lease, never plain --force
cd /tmp/[brief-id]
git fetch origin && git rebase origin/main
# resolve Cargo.toml workspace.members, etc.
git rebase --continue
git push --force-with-lease origin [branch]
```

The 500-LOC budget

Hard constraint enforced by every worker's acceptance checklist and verified by the conductor on review. Encourages splitting across sibling files (`worker_actions_executor.rs` + `worker_actions_executor_tests.rs` + `worker_actions_dispatch.rs`) rather than monolithing. Compounding effect: smaller diffs, faster review, less merge surface area.

9. The numbers (SKYCOT v1, end of run)

22	~21	~60 min	~50
BRIEF SLICES	PRS MERGED	WALL-CLOCK	SOURCE FILES ADDED
28	**<500**	**100%**	**1**
CRATES TOUCHED	LOC PER FILE	TESTS GREEN	CONDUCTOR TAB

Throughput pacing — what the swarm actually felt like

Phase	Wall time	What happened
Bootstrap	~5 min	Stream A executor trait + manifest enum lands first; unblocks B / C / G fan-out
Fan-out	~25 min	5 connectors (gmail / slack / gcal / hubspot / notion) and 4 channels (email / slack / sms / push / webinbox) merge in parallel waves
Identity + memory	~10 min	E-identity then F-memory-layering serial chain; sqlite-vec + Voyage-3 embeddings land
Capstone	~20 min	Stream H sub-sliced into H-1 / H-2 / H-3 / H-4a / H-4b / H-4c; serial in one tab, conductor merges each before next starts

Cadence rules of thumb

- **Brief size**: 1 brief is roughly 1 PR is roughly 200–500 LOC is roughly 30–60 minutes of one worker tab.
- **Conductor cadence**: 5-min tick fires 12 times an hour. About 50 percent no-ops, 30 percent merge one PR, 15 percent merge multiple or unblock, 5 percent require human escalation.
- **Worker concurrency**: 4 to 6 active workers is the sweet spot for one human operator. Above 8, the conductor's review queue becomes the bottleneck.
- **Throughput**: 1 PR every 3 minutes when the dependency graph is fanned-out. Closer to 1 every 5–8 minutes once serial dependencies dominate.

> *If the swarm stalls (no merges in two consecutive ticks, no open PRs, multiple BLOCKED rows): the conductor or human should cut a new tighter brief or split a fat one. The swarm runs on a steady diet of small briefs; starve it and it dies.*

10. The four artifacts (everything you need to bootstrap)

Artifact	Lines	Role
`BUILD_STATUS.md`	~100	The ledger. One table, one row per brief, three appendix sections (deps graph / blocked / done). Every state transition is a one-line commit.

Artifact	Lines	Role
`streams/[brief-id].md`	~50–150 each	One brief per row. Owner / depends-on / scope / files-may-touch / files-must-not-touch / acceptance-criteria / branch+PR conventions.
`CONTRIBUTING_AGENT.md`	~140	Worker-side protocol. Each worker pastes this into context once per session. Gates before opening a PR (build / lint / test / LOC budget); rules-that-override-everything.
`PROTOCOL_LOOP_PROMPT.md`	~95	The conductor-tick prompt pasted into `/loop 5m`. Nine numbered steps; step 8 is queue replenishment (writes fresh briefs when a TODO queue runs ≤3). Stages only BUILD_STATUS.md explicitly to prevent the webinbox bug.

Bootstrapping a new project (8 steps)

1. Create `docs/build-plan/` in your repo.
2. Copy the four template files into it.
3. Rename templates: `PROTOCOL_BUILD_STATUS_TEMPLATE.md` becomes `BUILD_STATUS.md`; fill in your brief rows.
4. Write one brief per intended PR slice in `streams/`. Tighten *may-touch* lists; the conductor will expand later.
5. Adapt `CONTRIBUTING_AGENT.md` to your toolchain (replace `cargo` with your build commands; keep the LOC budget).
6. Open one Claude tab. Paste `PROTOCOL_LOOP_PROMPT.md` into a `/loop 5m` command, substituting your repo path. Hit enter — the conductor is now running.
7. Open N worker tabs (mix Claude + Codex). In each: *Read CONTRIBUTING_AGENT.md and pick the first TODO row matching your agent family.*
8. Watch the ledger. Spawn fresh worker tabs as ones finish.

Why it works (in one breath)

- **First-commit-wins on BUILD_STATUS.md is sufficient.** `git push` is the lock; the loser pulls and picks a different row. Free correctness, ~5 races per hour, zero human action.
- **The per-brief may-touch list prevents most conflicts.** The hot path of shared seams (Cargo.toml / Cargo.lock / lib.rs / mod.rs) is either explicitly opened in the brief or hand-merged by the conductor in roughly 30 seconds.
- **Sub-slicing keeps the integration capstone tractable.** Stream H went from one fat brief to six sub-briefs; one tab landed all six serially while the conductor merged each before the next started.
- **No central registry.** Branch prefix carries family, `Co-Authored-By` trailer carries provenance, the owner column says who picks. That's it.

Companion to `docs/build-plan/PROTOCOL.md`. Diagrams render Mermaid v10. Source: SKYCOT v1 build of `argo`; ledger: `docs/build-plan/BUILD_STATUS.md`; history: `git log --oneline origin/main`.